Ultra-Wideband Indoor Positioning System

Design Document: Motivation, Objectives, Methodology & Mathematics

Target Platform: **Makerfabs ESP32 UWB Pro (DW1000)**

Sensing: **UWB Ranging + BNO085 IMU**

Output: **XYZ Position + Roll/Pitch/Yaw Orientation**

Version	2.0
Date	June 2026
Status	Active Development

Contents

1 Introduction and Motivation

1.1 Problem Statement

Accurate indoor positioning is a prerequisite for autonomous robotics, asset tracking, human-motion capture, augmented reality, and industrial safety systems. The Global Navigation Satellite System (GNSS) fails indoors because satellite signals are attenuated by building materials and the reflected multipath environment renders pseudorange measurements unreliable. Optical systems (cameras, LiDAR) achieve high accuracy but require significant infrastructure, computational resources, and are sensitive to lighting conditions and occlusion. Inertial systems drift unboundedly without external correction.

The result is a gap: no low-cost, infrastructure-light technology currently provides real-time 3D position *and* 3D orientation (six degrees of freedom, 6-DOF) indoors at centimetre-level accuracy with off-the-shelf microcontroller hardware.

1.2 Ultra-Wideband as a Candidate Technology

Ultra-wideband (UWB) radio occupies a spectrum of at least 500 MHz bandwidth (typically 3.1–10.6 GHz for IEEE 802.15.4a devices). The key physical properties that make UWB suitable for ranging are:

- **Time resolution.** The large bandwidth produces a narrow autocorrelation peak. The Decawave DW1000 timestamps signal arrivals at 15.65 ps resolution (64 GHz system clock), corresponding to a theoretical distance resolution of $15.65 \times 10^{-12} \times 3 \times 10^{8} \approx 4.7$ mm per clock tick.
- **Multipath separation.** The short pulse duration (∼2 ns) allows the receiver to distinguish the direct-path component from reflections that arrive within a few nanoseconds. Narrowband systems (Bluetooth, Wi-Fi, Zigbee) cannot make this separation.
- **Material penetration.** UWB signals propagate through common building materials (drywall, wood, glass), enabling ranging through soft obstructions at the cost of a positive distance bias.
- **Low power.** The FCC Part 15 power limit (−41.3 dBm/MHz) combined with the wide bandwidth results in pulse energy well below noise floor for narrowband receivers, making UWB inherently covert and interference-resistant.

The Decawave DW1000 chip, used in the Makerfabs ESP32 UWB Pro module, implements IEEE 802.15.4-2011 UWB and achieves a specified ranging accuracy of ±10 cm in line-of-sight (LOS) conditions.

1.3 Limitations of Existing Open-Source Libraries

Five open-source Arduino libraries implement DW1000 positioning. All share structural limitations that prevent scaling beyond four anchors or a single mobile tag:

The four-anchor ceiling in `thotro` and its derivatives arises from the RANGE frame encoding: each device occupies 17 bytes (2-byte address + three 5-byte absolute timestamps) within a 90-byte frame buffer. With the 9-byte MAC header, only $\lfloor (90 - 9 - 2)/17 \rfloor = 4$ devices fit before buffer overflow. The `pizzo00` fork reduces this to 12 bytes/device using differential timestamps, reaching 6 anchors, but the broadcast POLL mechanism still causes RF collision between anchor replies.

Critically, *no existing library* uses the DW1000's first-path power registers for non-line-of-sight (NLOS) detection, despite the chip making this data available on every received frame.

Table 1: Open-source DW1000 library comparison

Feature	thotro	Makerfabs	jremington	pizzo00	dw1000-ng
Anchor ceiling	4	4	7*	6	Unlimited
RF collisions	Yes	Yes	Yes	Yes	No
Update rate (Hz)	0.6	0.6	0.6	1	2
Multi-tag	No	No	No	Partial	No
NLOS detection	No	No	No	No	No
3D positioning	No	No	Fragile	No	No
IMU fusion	No	No	No	No	No
Maintained	No	Partial	Yes	Yes	No

1.4 Proposed Contribution

This project designs and implements a scalable UWB real-time locating system (RTLS) with the following distinguishing characteristics:

1. **Unlimited anchor scaling** via individual addressed DS-TWR exchanges (no shared frame buffer, no broadcast collisions).
2. **NLOS detection** using the DW1000 first-path power registers (FP_AMPL1/2/3) — the first open-source implementation of this capability.
3. **Physics-aware state estimation** using a Moving Horizon Estimator with Huber loss, physical room-boundary constraints, and speed limits.
4. **Full 6-DOF output** (XYZ position + RPY orientation) via tight fusion of UWB ranging and BNO085 IMU data.
5. **Anchor self-survey** via anchor-to-anchor ranging and Multidimensional Scaling, eliminating the need for manual coordinate measurement.

2 System Overview

2.1 Hardware

Hardware Specification

UWB module	Makerfabs ESP32 UWB Pro with Display
UWB chip	Decawave DW1000 (IEEE 802.15.4-2011 UWB)
MCU	ESP32 (dual-core Xtensa LX6, 240 MHz, Wi-Fi/BT)
RF front-end	PA/LNA amplifier (∼200 m outdoor range)
Display	1.3 inch SSD1306 OLED (I^2C, pins SDA=4, SCL=5)
IMU (planned)	Bosch BNO085 via SPI
Configuration	3–4 anchors (fixed) + 1 tag (mobile)
Transport	Wi-Fi UDP to MATLAB host (192.168.x.x:5005)

2.2 Software Architecture

The system is split into two layers:

Firmware (Arduino/ESP32): The tag performs round-robin DS-TWR ranging to each anchor, collects raw distances with quality metrics, and streams measurement packets over UDP. Anchors are purely passive responders. The tag never solves its own position.

Host (MATLAB): All position computation, filtering, visualisation, and logging runs on the host PC. This separation means adding anchors requires only editing a JSON configuration file — no firmware change. The firmware packet format is versioned and IMU-ready.

The processing pipeline on the host is:

$$\text{Raw ranges} \xrightarrow{\text{median filter}} \text{Filtered ranges} \xrightarrow{\text{NLOS weighting}} \text{Weighted ranges} \xrightarrow{\text{LM multilat.}} \text{Position} \xrightarrow{\text{EKF/MHE}} \hat{\boldsymbol{x}} \xrightarrow{\text{EMA}}$$

3 Objectives

O1. Achieve XY positioning accuracy of $\leq 3\,\text{cm}$ (1-σ) in LOS and $\leq 8\,\text{cm}$ in soft NLOS conditions using 3–4 anchors.

O2. Achieve Z-axis positioning accuracy of $\leq 5\,\text{cm}$ with 4 anchors deployed at different heights.

O3. Deliver continuous RPY orientation output at $\geq 50\,\text{Hz}$ from BNO085 IMU fusion.

O4. Maintain position tracking continuity during brief anchor occlusions ($\leq 3\,\text{s}$) using IMU dead-reckoning.

O5. Support anchor self-survey: determine anchor positions to $\leq 2\,\text{cm}$ without external measurement.

O6. Demonstrate update rate $\geq 3\,\text{Hz}$ for 3 anchors and $\geq 8\,\text{Hz}$ in fast-mode operation.

O7. Provide real-time NLOS detection per anchor using DW1000 channel impulse response diagnostics.

O8. Remain compatible with low-cost hardware: total cost $<$ \$200 for a 4-anchor + 1-tag system.

4 Mathematical Foundations

This section develops the mathematics behind each processing stage in the pipeline. No prior knowledge of signal processing or estimation theory is assumed. **Notation:** Scalars are written in italic (t, d, τ); column vectors in bold italic ($\boldsymbol{x}$, $\boldsymbol{p}$); matrices in bold upright ($\mathbf{F}$, $\mathbf{Q}$). The superscript $^\top$ means transpose (turning a column into a row). $\|\boldsymbol{v}\|$ is the length of vector $\boldsymbol{v}$ (square root of sum of squares of its components). Subscripts k index discrete time steps.

4.1 Double-Sided Two-Way Ranging (DS-TWR)

We want to measure the distance between two radio modules by timing how long a signal takes to travel between them. The speed of light is known ($c = 3 \times 10^8\,\text{m/s}$), so distance $= c \times$ time. The problem is that the clock inside each module runs at a slightly different speed. Even a tiny difference — say 20 parts per million (ppm), which is 20 microseconds per second — causes a large range error. If the tag waits 1 ms before replying to an anchor, a 20 ppm clock error makes it *think* 1 ms has passed when in reality $1\,\text{ms} \times (1 + 20 \times 10^{-6}) = 1.00002\,\text{ms}$ has passed. That extra $0.00002\,\text{ms}$ corresponds to $0.00002 \times 10^{-3} \times 3 \times 10^8 = 6\,\text{m}$ of phantom range. Catastrophic.
DS-TWR fixes this by performing a two-sided exchange so that the clock offsets cancel algebraically — without either device needing to know its clock error.

4.1.1 Message Exchange

The DS-TWR protocol uses four messages between a tag (T) and anchor (A), each recorded with a hardware timestamp:

Step	Direction	Frame	Timestamps captured
1	T → A	POLL	t_1 (T transmits), t_2 (A receives)
2	A → T	POLL-ACK	t_3 (A transmits), t_4 (T receives)
3	T → A	RANGE	t_5 (T transmits); carries t_1, t_4, t_5
4	A → T	RANGE-REPORT	t_6 (A receives); anchor computes & replies d

Think of it as a conversation: the tag says "hello" (t_1), the anchor hears it (t_2) and replies (t_3), the tag hears the reply (t_4) and sends a follow-up (t_5), and the anchor hears that (t_6). Both sides now have enough timing information to eliminate the clock offset.

4.1.2 Defining the Timing Intervals

From these six timestamps we form four intervals (all measured in DW1000 hardware ticks; one tick = 15.65 ps):

$$R_a = t_4 - t_1 \quad \text{tag's round-trip time (poll sent → ack received)} \tag{1}$$

$$D_a = t_3 - t_2 \quad \text{anchor's reply delay (poll received → ack sent)} \tag{2}$$

$$R_b = t_6 - t_3 \quad \text{anchor's round-trip time (ack sent → range received)} \tag{3}$$

$$D_b = t_5 - t_4 \quad \text{tag's reply delay (ack received → range sent)} \tag{4}$$

If there were no clock error and the signal takes time τ to travel one way, then ideally $R_a = 2\tau + D_a$ and $R_b = 2\tau + D_b$. Each round-trip is twice the travel time plus the processing delay.

4.1.3 The DS-TWR Formula

Combining both round-trips cancels the processing delays and, crucially, the clock offsets:

$$\boxed{\tau_{\text{ToF}} = \frac{R_a R_b - D_a D_b}{R_a + R_b + D_a + D_b}} \tag{5}$$

Consider the ideal case (no clock error). Substitute $R_a = 2\tau + D_a$ and $R_b = 2\tau + D_b$:

$$\tau_{\text{ToF}} = \frac{(2\tau + D_a)(2\tau + D_b) - D_a D_b}{(2\tau + D_a) + (2\tau + D_b) + D_a + D_b} = \frac{4\tau^2 + 2\tau(D_a + D_b)}{4\tau + 2(D_a + D_b)} = \frac{2\tau\big(2\tau + D_a + D_b\big)}{2\big(2\tau + D_a + D_b\big)} = \tau$$

The processing delays D_a and D_b cancel completely, leaving the true one-way travel time. With clock errors, the algebra is more involved but the errors cancel to second order.

Clock-offset cancellation (detailed). Let ε_T and ε_A denote the fractional frequency offsets of the tag and anchor clocks (e.g. $\varepsilon = 20 \times 10^{-6}$ for a 20 ppm crystal). Because the tag's clock runs fast or slow, all intervals measured by the tag are scaled: R_a (measured by the tag) is really $(2\tau + D_a)(1 + \varepsilon_T)$, and R_b (measured by the anchor) is $(2\tau + D_b)(1 + \varepsilon_A)$. Substituting into

eq. (5) and keeping only first-order terms in ε (since $\varepsilon^2 \approx 4 \times 10^{-10}$ is negligible):

$$\tau_{\text{ToF}} \approx \tau \quad + \quad \underbrace{O(\varepsilon^2)}_{\approx 0.1\,\text{mm}}$$

The clock-offset terms vanish. The remaining error is at most 0.1 mm — far below the chip's 4.7 mm resolution.

Worked numerical example

Suppose a tag ranges to an anchor 3 m away. Light travel time: $\tau = 3 \,/\, (3 \times 10^8) = 10\,\text{ns} = 10 \times 10^{-9}\,\text{s}$. In DW1000 ticks (1 tick = 15.65 ps), this is $10 \times 10^{-9} \,/\, 15.65 \times 10^{-12} \approx 639$ ticks. Say the anchor takes 1 ms to process and reply ($D_a = 1\,\text{ms}$), and the tag takes 1.2 ms ($D_b = 1.2\,\text{ms}$). In ticks:

$$D_a \approx 63\,900\,000, \quad D_b \approx 76\,700\,000$$

$$R_a = 2\tau + D_a \approx 1278 + 63\,900\,000 = 63\,901\,278 \text{ ticks}$$

$$R_b = 2\tau + D_b \approx 1278 + 76\,700\,000 = 76\,701\,278 \text{ ticks}$$

$$\tau_{\text{ToF}} = \frac{R_a R_b - D_a D_b}{R_a + R_b + D_a + D_b} \approx \frac{63901278 \times 76701278 - 63900000 \times 76700000}{63901278 + 76701278 + 63900000 + 76700000}$$

Numerically the cross-terms from $D_a D_b$ and the products cancel to leave ≈ 639 ticks, giving: $\hat{d} = 639 \times 15.65 \times 10^{-12} \times 3 \times 10^8 \approx 3.0\,\text{m}$. ✓

The measured distance is then:

$$\hat{d} = c \cdot \tau_{\text{ToF}} + b_{\text{ant}} \tag{6}$$

where $c = 2.998 \times 10^8\,\text{m/s}$ and b_{ant} is a per-board calibration offset (see below).

4.1.4 Antenna Delay Calibration

Every DW1000 chip has a slightly different internal wiring delay between the digital timestamp and the physical antenna. This delay is fixed for a given board but unknown. Without correcting it, every range measurement will be off by the same constant — even with perfect DS-TWR maths. Calibration identifies this constant by comparing the measured distance against a known true distance and adjusting a register (Δ) until they match.

The DW1000 applies a programmable antenna delay Δ (a 16-bit register value) that shifts the RX timestamp. Increasing Δ makes the chip think the signal arrived later, which reduces the computed ToF and thus the reported distance. Calibration finds:

$$\Delta^* = \arg\min_{\Delta} \left(\mathbb{E}\left[\hat{d}(\Delta)\right] - d^\star \right)^2$$

where $d^\star$ is the measured true distance (e.g. boards placed 7 m apart with a tape measure). The relationship is monotone, so a binary search over $\Delta \in [15800, 16900]$ with 200 averaged measurements per candidate converges to the optimum in 14 iterations ($2^{14} = 16384$ covers the range). The residual uncertainty is ± 3 ticks $\approx \pm 1.4$ mm.

4.2 NLOS Detection via Channel Impulse Response

When the direct radio path between tag and anchor is blocked by a wall or large object, the signal still arrives — but via a longer reflected route. Since we assume straight-line travel, the extra path length registers as a spurious positive distance error (the measured range is longer than the true range). This is called Non-Line-of-Sight (NLOS) error and is always positive.
The DW1000 gives us a way to detect NLOS without any extra hardware: it separately reports how much energy arrived in the *first* path (the direct or near-direct arrival) versus the *total* received energy. If a lot of energy came in late (via reflections), the ratio between total and first-path power is large — a reliable NLOS indicator.

4.2.1 Physical Basis

When a radio pulse is transmitted, it spreads out in space. At the receiver, the DW1000 correlates the incoming signal against the known preamble sequence to build a *Channel Impulse Response* (CIR) — essentially a time-series showing when energy arrives and how much. In LOS conditions, one sharp peak appears at the direct-path delay. In NLOS, additional energy arrives later via reflected paths. The chip hardware captures the amplitude of the three samples straddling the first (earliest) peak, giving us a window into how strong the direct path is relative to all the scattered energy.

4.2.2 DW1000 Diagnostic Registers

After every received frame, the DW1000 populates these registers automatically:

- **FP_AMPL1, FP_AMPL2, FP_AMPL3** (register 0x12): three amplitude samples around the first-path leading edge. Together they represent the energy in the direct (first-arriving) signal.
- **RXPACC** (register 0x10): the preamble accumulator count — how many samples were averaged to build the CIR. Used as a normalisation factor.
- **CIR_PWR** (register 0x12): the total integrated channel power, including all reflections.

4.2.3 Computing the NLOS Score

Step 1 — First-path power. Combine the three amplitude samples and normalise by the accumulator count. The result is then converted to decibels (dBm), a logarithmic power scale where each 10 dB corresponds to a factor of 10 in power:

$$P_{\mathrm{FP}} = 10\log_{10}\left(\frac{A_1^2 + A_2^2 + A_3^2}{N_{\mathrm{acc}}^2}\right) - A_{\mathrm{const}} \tag{7}$$

Here A_1, A_2, A_3 are the FP_AMPL values, N_{acc} is RXPACC, and A_{const} is a chip-specific constant (121.74 dBm at 16 MHz PRF; 113.77 dBm at 64 MHz PRF) that accounts for the chip's internal gain.

Step 2 — Total received power. The total power P_{RX} is read from CIR_PWR via the driver function `DW1000.getReceivePower()`, using a similar log formula.

Step 3 — NLOS score. The NLOS score is simply the power gap between total and first-path:

$$\boxed{\Delta P_{\mathrm{NLOS}} = P_{\mathrm{RX}} - P_{\mathrm{FP}}} \tag{8}$$

In LOS, most of the energy arrives in the first path, so $P_{\mathrm{RX}} \approx P_{\mathrm{FP}}$ and ΔP_{NLOS} is small. In NLOS, extra reflected energy raises P_{RX} while P_{FP} stays low, widening the gap.

Score	Condition	Action
$< 3\,\mathrm{dB}$	LOS. Direct path dominates.	Full weight in multilateration.
3–6 dB	Soft NLOS. Partial obstruction.	Reduce weight proportionally.
$\geq 6\,\mathrm{dB}$	Hard NLOS. Reflected path dominant.	Strong downweight or reject.

4.2.4 Range Bias Correction under NLOS

NLOS ranging error is *always positive*: reflected and through-wall paths are geometrically longer than the direct path, so the measured distance always exceeds the true distance. Through-wall propagation also slows the signal by a factor of $\sqrt{\varepsilon_r}$, where ε_r is the material's relative permittivity (concrete: $\varepsilon_r \approx 6$–9; drywall: $\varepsilon_r \approx 2$–4). A higher ΔP_{NLOS} score correlates with a larger positive bias, allowing an empirical correction:

$$b_{\mathrm{NLOS}} \approx k_{\mathrm{NLOS}} \cdot \max(0,\ \Delta P_{\mathrm{NLOS}} - 3\,\mathrm{dB}) \tag{9}$$

with $k_{\mathrm{NLOS}} \approx 0.02$–$0.05\,\mathrm{m/dB}$ (environment-dependent, tuned experimentally). This correction is subtracted from the measured range before multilateration.

4.3 Weighted Multilateration

Multilateration is the process of finding a position given distances to several reference points (anchors). The key idea: if you know you are exactly 3 m from anchor 1, you could be anywhere on a circle (or sphere in 3D) of radius 3 m centred on anchor 1. A second anchor gives a second circle. In 2D, two circles intersect at two points. A third anchor breaks the ambiguity and gives a unique intersection — your position.
In practice, the measured distances have noise, so the circles do not intersect perfectly at one point. Instead of looking for an exact intersection, we find the position that is *as close as possible* to being at the stated distance from each anchor — a least-squares problem.

4.3.1 Measurement Model

Let there be N anchors at *known* positions $\boldsymbol{a}_i \in \mathbb{R}^d$ (where $d = 2$ for 2D or $d = 3$ for 3D), and let $\hat{d}_i$ be the measured range to anchor i after median filtering and bias correction. The true tag position $\boldsymbol{x} \in \mathbb{R}^d$ satisfies:

$$\hat{d}_i = \underbrace{\|\boldsymbol{x} - \boldsymbol{a}_i\|}_{\text{true distance}} + \underbrace{e_i}_{\text{noise}}, \quad e_i \sim \mathcal{N}(0, \sigma_i^2) \tag{10}$$

The term $\|\boldsymbol{x} - \boldsymbol{a}_i\|$ is the Euclidean distance from tag to anchor i (the length of the straight line connecting them). The noise e_i is modelled as Gaussian with standard deviation σ_i. Anchors flagged as NLOS have a larger σ_i:

$$\sigma_i = \sigma_{\mathrm{base}} \cdot 10^{\Delta P_{\mathrm{NLOS},i}/20} \tag{11}$$

A 10 dB NLOS score multiplies the assumed noise standard deviation by $10^{0.5} \approx 3.16$, reducing the weight (see below) by a factor of 10. This is how the NLOS score flows from the radio measurement into the position solver.

4.3.2 Weighted Least-Squares Objective

Define the *residual* for anchor i as $r_i(\boldsymbol{x}) = \|\boldsymbol{x} - \boldsymbol{a}_i\| - \hat{d}_i$ — the difference between the distance implied by guess $\boldsymbol{x}$ and the measured range. If our guess is correct, $r_i = 0$. The position estimate minimises the weighted sum of squared residuals:

$$\hat{\boldsymbol{x}} = \arg\min_{\boldsymbol{x}\in\mathbb{R}^d} \sum_{i=1}^{N} w_i\, r_i(\boldsymbol{x})^2, \quad w_i = \frac{1}{\sigma_i^2} \tag{12}$$

Anchors with higher uncertainty σ_i get a smaller weight w_i, so their contribution to the cost is reduced. This is the core of *weighted least squares*: trust reliable measurements more.

4.3.3 Levenberg-Marquardt Solver

The objective eq. (12) is nonlinear (the distances $\|\boldsymbol{x} - \boldsymbol{a}_i\|$ are nonlinear in $\boldsymbol{x}$), so there is no closed-form solution. We use the Levenberg-Marquardt (LM) algorithm, which iteratively improves a guess $\boldsymbol{x}_k$ by solving a linearised sub-problem at each step.

The Jacobian J. At the current guess $\boldsymbol{x}_k$, we compute the Jacobian matrix $\mathbf{J} \in \mathbb{R}^{N\times d}$. Each row is the gradient of one residual with respect to the position:

$$\mathbf{J}_i = \frac{\partial r_i}{\partial \boldsymbol{x}} = \frac{(\boldsymbol{x} - \boldsymbol{a}_i)^\top}{\|\boldsymbol{x} - \boldsymbol{a}_i\|} \tag{13}$$

This is just the unit vector pointing from anchor i toward the current guess $\boldsymbol{x}$. Intuitively, it says: "if you move in direction $\mathbf{J}_i$, the distance to anchor i increases by one unit."

LM update step. Let $\mathbf{W} = \mathrm{diag}(w_1, \ldots, w_N)$ be the diagonal weight matrix. The LM correction to the current guess is:

$$\boldsymbol{x}_{k+1} = \boldsymbol{x}_k - \left(\mathbf{J}^\top\mathbf{W}\mathbf{J} + \lambda\,\mathrm{diag}(\mathbf{J}^\top\mathbf{W}\mathbf{J})\right)^{-1}\mathbf{J}^\top\mathbf{W}\boldsymbol{r}(\boldsymbol{x}_k) \tag{14}$$

The damping parameter λ controls how cautious each step is. When λ is large the step shrinks toward the direction of steepest descent (safe but slow). When λ is small the step is a full Newton step (fast near the solution). LM automatically adjusts λ: increase it if the step made things worse, decrease it if it made things better. This gives it the robustness of gradient descent and the speed of Newton's method near the solution.

4.3.4 Robust Outlier Gating (MAD)

A single badly-NLOS anchor can dominate the cost and pull the estimate far from the truth. After a first LM solve, outlier anchors are detected using the *Median Absolute Deviation* (MAD):

$$\hat{\sigma}_{\text{robust}} = 1.4826 \cdot \mathrm{median}(|r_i - \mathrm{median}(\boldsymbol{r})|) \tag{15}$$

The regular standard deviation is sensitive to outliers: one anchor with a 2 m NLOS error pulls the mean residual and inflates the standard deviation, masking the fact that other anchors are perfectly consistent. The *median* ignores the extreme values and reports the typical residual. The constant 1.4826 converts the MAD to an equivalent standard deviation under Gaussian noise. Anchors whose residual deviates from the median by more than $K = 2.5$ times this robust sigma are excluded and the solve repeats with the remaining anchors.

4.3.5 Position Covariance

The uncertainty in the position estimate (needed by the Kalman filter) is approximated from the curvature of the cost function at the solution:

$$\mathbf{P}_{\text{pos}} \approx \sigma_{\text{base}}^2 \left(\mathbf{J}^\top \mathbf{W} \mathbf{J}\right)^{-1} \tag{16}$$

Intuitively: if all the Jacobian rows point in similar directions (anchors roughly collinear), the matrix $\mathbf{J}^\top \mathbf{W} \mathbf{J}$ is nearly singular and its inverse is large — meaning the position is poorly determined in some directions. If the anchors surround the tag from all sides, the matrix is well-conditioned and the position uncertainty is small. This is the mathematical expression of the "Dilution of Precision" concept familiar from GPS.

4.3.6 GPS-Style Simultaneous Bias Estimation

In GPS, the receiver's clock is imprecise, so every pseudorange measurement has the same unknown clock offset added to it. GPS solves for position *and* this offset simultaneously by using one extra satellite beyond the minimum. The offset term cancels in pairwise differences between satellites, leaving only geometry.
We face the same problem: an uncalibrated or temperature-drifted tag clock adds the same offset δ to all measured ranges. With $N \geq d + 2$ anchors (one extra beyond the minimum needed for position), we can estimate δ alongside position.

The augmented measurement model is:

$$\hat{d}_i = \|\boldsymbol{x} - \boldsymbol{a}_i\| + \delta + e_i \tag{17}$$

The augmented Jacobian gains an extra column of ones (because moving δ by 1 shifts all residuals by 1):

$$\tilde{\mathbf{J}}_i = [\mathbf{J}_i,\ 1] \tag{18}$$

The LM solver treats $\tilde{\boldsymbol{x}} = [\boldsymbol{x}^\top, \delta]^\top$ as the unknown and proceeds identically.

Geometric limitation with minimum anchors. If we try this with exactly $d + 1$ anchors (e.g. 3 anchors in 2D), the δ estimate is not reliable. Subtracting any two range equations cancels δ exactly, so all the information about position comes from differences, not absolute ranges. The solver becomes ill-conditioned near the anchor centroid. For this reason, δ-solve is only enabled when $N \geq d + 2$.

4.4 State Estimation

4.4.1 Extended Kalman Filter (EKF)

The multilateration solver gives a new position estimate every time a ranging sweep completes (roughly 3 Hz). The Kalman filter is a principled way to *smooth* these noisy position estimates using knowledge of how things physically move.
The filter alternates between two steps every time interval Δt:

1. **Predict.** "Given where the tag was and how fast it was moving, where should it be now?" The filter projects the state forward in time using Newton's equations.
2. **Update.** "A new measurement arrived. How much should I correct my prediction?"

The correction is proportional to how much we trust the measurement versus the physics model.

The filter tracks not just the state estimate but also its *uncertainty* (the covariance matrix $\mathbf{P}$). Uncertainty grows during prediction (we are less sure of where the tag is without measurements) and shrinks during update (new data reduces uncertainty).

State vector. The EKF state packs all unknowns into a single column vector:

$$\boldsymbol{x} = \begin{bmatrix} \boldsymbol{p} \\ \boldsymbol{v} \\ \boldsymbol{a} \\ \delta \end{bmatrix} \in \mathbb{R}^{3d+1} \tag{19}$$

where $\boldsymbol{p} \in \mathbb{R}^d$ is position (e.g. $[x, y]$ in 2D), $\boldsymbol{v} \in \mathbb{R}^d$ is velocity (how fast position is changing), $\boldsymbol{a} \in \mathbb{R}^d$ is acceleration (how fast velocity is changing), and $\delta \in \mathbb{R}$ is the slowly-varying range bias from §4.3.5.

Transition matrix F (predict step). The state evolves according to Newton's kinematic equations. If the acceleration stays roughly constant over one time step Δt, then:

$$p_{\text{new}} = p + v\,\Delta t + \tfrac{1}{2} a\,\Delta t^2, \quad v_{\text{new}} = v + a\,\Delta t, \quad a_{\text{new}} \approx a, \quad \delta_{\text{new}} \approx \delta$$

Written in matrix form for the full state vector:

$$\mathbf{F} = \begin{bmatrix} \mathbf{I}_d & \Delta t\,\mathbf{I}_d & \frac{\Delta t^2}{2}\mathbf{I}_d & \mathbf{0} \\ \mathbf{0} & \mathbf{I}_d & \Delta t\,\mathbf{I}_d & \mathbf{0} \\ \mathbf{0} & \mathbf{0} & \mathbf{I}_d & \mathbf{0} \\ 0 & 0 & 0 & 1 \end{bmatrix} \tag{20}$$

Each block row corresponds to one sub-state (position, velocity, acceleration, bias). The top-left 3×3 block in $d = 1$ reads: new position = old position $\times 1$ + old velocity $\times \Delta t$ + old acceleration $\times \Delta t^2/2$. This is just kinematics.

Process noise Q (model uncertainty). We know the tag does not follow perfect constant-acceleration motion. Random forces (sudden stops, direction changes) introduce unpredictable acceleration changes. The matrix $\mathbf{Q}$ models this uncertainty. Larger $\mathbf{Q}$ means we trust the physics model less and will weight new measurements more heavily:

$$\mathbf{Q} = q_j \begin{bmatrix} \frac{\Delta t^5}{20}\mathbf{I} & \frac{\Delta t^4}{8}\mathbf{I} & \frac{\Delta t^3}{6}\mathbf{I} & \mathbf{0} \\ \frac{\Delta t^4}{8}\mathbf{I} & \frac{\Delta t^3}{3}\mathbf{I} & \frac{\Delta t^2}{2}\mathbf{I} & \mathbf{0} \\ \frac{\Delta t^3}{6}\mathbf{I} & \frac{\Delta t^2}{2}\mathbf{I} & \Delta t\,\mathbf{I} & \mathbf{0} \\ \mathbf{0} & \mathbf{0} & \mathbf{0} & q_\delta\,\Delta t \end{bmatrix} \tag{21}$$

The scalar q_j (jerk power spectral density, tuned via the `EKF_Q_ACCEL` slider) controls the trade-off: *low* q_j makes the filter trust its kinematic model and produce smooth but potentially lagging output; *high* q_j makes it follow measurements closely but with more noise.

Measurement model for tight coupling. Rather than feeding the multilaterated position directly into the filter, we can feed the raw ranges and let the filter compute the expected ranges from its own state estimate. For anchor i at position $\boldsymbol{a}_i$, the predicted range is:

$$h_i(\boldsymbol{x}) = \|\boldsymbol{p} - \boldsymbol{a}_i\| + \delta \tag{22}$$

The linearised measurement Jacobian (gradient of h_i with respect to the state) is:

$$\mathbf{H}_i = \left[\underbrace{\frac{(\boldsymbol{p} - \boldsymbol{a}_i)^\top}{\|\boldsymbol{p} - \boldsymbol{a}_i\|}}_{\text{unit vector toward anchor}} , \underbrace{\mathbf{0}_{1\times 2d}}_{\text{vel/acc terms}} , \underbrace{1}_{\text{bias term}} \right] \tag{23}$$

The first part is the unit vector pointing from the tag toward anchor i; moving the tag by 1 m in that direction changes the range to anchor i by exactly 1 m.

Kalman equations. At each time step the filter runs the following sequence:

Predict:
$$\hat{\boldsymbol{x}}_{k|k-1} = \mathbf{F}\hat{\boldsymbol{x}}_{k-1|k-1} \quad \text{(project state forward)} \tag{24}$$
$$\mathbf{P}_{k|k-1} = \mathbf{F}\mathbf{P}_{k-1|k-1}\mathbf{F}^\top + \mathbf{Q} \quad \text{(uncertainty grows with time)} \tag{25}$$

Update:
$$\mathbf{S}_k = \mathbf{H}_k\mathbf{P}_{k|k-1}\mathbf{H}_k^\top + \mathbf{R}_k \quad \text{(innovation covariance)} \tag{26}$$
$$\mathbf{K}_k = \mathbf{P}_{k|k-1}\mathbf{H}_k^\top\mathbf{S}_k^{-1} \quad \text{(Kalman gain)} \tag{27}$$
$$\hat{\boldsymbol{x}}_{k|k} = \hat{\boldsymbol{x}}_{k|k-1} + \mathbf{K}_k(\boldsymbol{z}_k - h(\hat{\boldsymbol{x}}_{k|k-1})) \quad \text{(correct estimate)} \tag{28}$$
$$\mathbf{P}_{k|k} = (\mathbf{I} - \mathbf{K}_k\mathbf{H}_k)\mathbf{P}_{k|k-1} \quad \text{(uncertainty shrinks after update)} \tag{29}$$

The Kalman gain $\mathbf{K}$ decides how much to correct the prediction when a measurement arrives. It is automatically computed from two competing uncertainties:

- $\mathbf{P}_{k|k-1}$: how uncertain we are in our *prediction*. Large $\mathbf{P}$ means "I am not confident where the tag is."
- $\mathbf{R}_k$: how uncertain we are in the *measurement* (the covariance from the multilaterator, eq. (16)). Large $\mathbf{R}$ means "I do not trust this measurement."

If the prediction is uncertain (large $\mathbf{P}$) and the measurement is reliable (small $\mathbf{R}$), then $\mathbf{K}$ is large: the filter corrects strongly toward the measurement. If the prediction is confident (small $\mathbf{P}$) and the measurement is noisy (large $\mathbf{R}$), $\mathbf{K}$ is small: the filter barely moves. This is the optimal weighted combination.

The subscript notation $\hat{\boldsymbol{x}}_{k|k-1}$ means "estimate of state at time k, using data up to time $k-1$" (prediction). $\hat{\boldsymbol{x}}_{k|k}$ means "estimate at time k, after incorporating measurement at time k" (updated).

4.4.2 Moving Horizon Estimator (MHE)

The Kalman filter processes one measurement at a time and maintains a compact summary (mean and covariance). The Moving Horizon Estimator takes a different approach: at every time step, it looks at the last N measurements simultaneously and solves one large optimisation problem to find the trajectory that best fits all of them together.
This is more computationally expensive but offers two key advantages:

1. **Hard constraints.** The solution is forced to lie inside the room and respect a speed limit. The Kalman filter cannot enforce these.
2. **Robustness to outliers.** Using the Huber loss (described below), the MHE naturally reduces the influence of NLOS-corrupted measurements without explicitly rejecting them.

At time step k, the MHE considers the window of past states $\{\boldsymbol{x}_{k-N+1}, \ldots, \boldsymbol{x}_k\}$ and minimises:

$$\min_{\{\boldsymbol{x}_t\}} \underbrace{\|\boldsymbol{x}_{k-N+1} - \bar{\boldsymbol{x}}_{k-N+1}\|^2_{\mathbf{P}_{\text{arr}}^{-1}}}_{\substack{\text{arrival cost} \\ \text{(memory of the past)}}} + \underbrace{\sum_{t=k-N+2}^{k} \|\boldsymbol{x}_t - \mathbf{F}\boldsymbol{x}_{t-1}\|^2_{\mathbf{Q}^{-1}}}_{\substack{\text{dynamics residuals} \\ \text{(penalises unphysical jumps)}}} + \underbrace{\sum_t \sum_i \rho_\delta\Big(\|\boldsymbol{p}_t - \boldsymbol{a}_i\| - \hat{d}_{t,i}\Big)}_{\substack{\text{range residuals} \\ \text{(fit the measurements)}}} \tag{30}$$

where $\|\boldsymbol{v}\|^2_{\mathbf{M}} = \boldsymbol{v}^\top \mathbf{M} \boldsymbol{v}$ is a weighted norm. The three terms penalise:

1. **Arrival cost:** deviation of the oldest state in the window from the summary $\bar{\boldsymbol{x}}_{k-N+1}$ carried forward from the previous window. This compactly encodes all information from before the window.
2. **Dynamics residuals:** how much each consecutive pair of states violates the physics model $\boldsymbol{x}_t \approx \mathbf{F}\boldsymbol{x}_{t-1}$. Large jumps are penalised.
3. **Range residuals:** how well the estimated positions fit the UWB range measurements, using the Huber loss ρ_δ.

The Huber loss ρ_δ. Standard least squares penalises errors quadratically: doubling the error quadruples the penalty. This means a single large NLOS error can dominate the whole objective. The Huber loss switches to a linear penalty for large errors, limiting the influence of any single outlier:

$$\rho_\delta(e) = \begin{cases} \dfrac{1}{2}e^2 & \text{if } |e| \le \delta \\ \delta\left(|e| - \dfrac{\delta}{2}\right) & \text{if } |e| > \delta \end{cases} \tag{31}$$

Residual $\lvert e\rvert$	**Squared loss**	**Huber loss ($\delta = 0.15\,\mathbf{m}$)**
0.05 m	0.0025	0.00125 (quadratic)
0.15 m	0.0225	0.01125 (quadratic, at threshold)
0.30 m	0.0900	0.01688 (linear; only 12× larger than 0.05 m)
1.00 m	1.0000	0.13500 (linear; only 108× larger than 0.05 m)

An NLOS anchor with a 1 m error contributes 108× less than a perfect anchor under Huber loss, versus 400× more under squared loss.

Constraints:

$$\boldsymbol{p}_{\min} \le \boldsymbol{p}_t \le \boldsymbol{p}_{\max} \qquad \text{(room boundary, hard constraint)} \tag{32}$$

$$\|\boldsymbol{v}_t\|_2 \le v_{\max} \qquad \text{(speed limit, hard constraint)} \tag{33}$$

These hard constraints prevent the estimator from placing the tag outside the room or moving it at superhuman speed, which can happen during anchor occlusions if the estimator is unconstrained.

Computational cost. With window $N = 6$, space $d = 3$, and $N_a = 4$ anchors, the problem has $N \times (3d + 1) = 60$ unknowns and is solved by `fmincon` (MATLAB interior-point method) in 2–5 ms per step — well inside the 333 ms between UWB sweeps.

Table 2: EKF vs. MHE comparison

Property	**EKF**	**MHE**
NLOS handling	Pre-gating required	Huber loss absorbs outliers
Physical constraints	None	Room boundary + speed (hard)
Nonlinearity	Jacobian linearisation	Solved exactly
Lag	Zero	$N/2$ steps ($\approx 1\,\text{s}$ at $N = 6$)
Compute per step	Trivial (μs)	2–5 ms (fmincon)
Sensitivity to initialisation	Moderate	Low (window smooths errors)

4.5 IMU Fusion and Orientation Estimation

UWB provides a position fix roughly every 300 ms. An IMU (inertial measurement unit) measures angular rates (gyroscope) and linear acceleration at 100 Hz. Between UWB updates, the IMU provides 30 intermediate position estimates using dead-reckoning (integrating acceleration twice). At each UWB update, the integrated position is corrected. Together, UWB provides the absolute reference while the IMU provides smooth high-rate motion between fixes.
The BNO085 also outputs orientation (roll/pitch/yaw) at 100 Hz directly, which UWB alone cannot provide.

4.5.1 Why Quaternions?

Orientation in 3D can be parameterised in several ways. The most intuitive is three Euler angles (roll ϕ, pitch θ, yaw ψ) — tilt left/right, tilt forward/back, and turn left/right. However, Euler angles have a singularity called *gimbal lock*: when the pitch angle reaches $\pm 90°$, roll and yaw become indistinguishable and one degree of freedom is lost. This causes numerical instability in tracking algorithms.

Quaternions avoid this by representing orientation as four numbers $[q_w, q_x, q_y, q_z]$ with the constraint $q_w^2 + q_x^2 + q_y^2 + q_z^2 = 1$. The geometric interpretation: every rotation can be described as a rotation by angle θ about a unit axis $[\hat{n}_x, \hat{n}_y, \hat{n}_z]$, encoded as:

$$q_w = \cos(\theta/2), \quad q_x = \hat{n}_x \sin(\theta/2), \quad q_y = \hat{n}_y \sin(\theta/2), \quad q_z = \hat{n}_z \sin(\theta/2)$$

For example, a 90° yaw (rotation about the Z axis) gives $\boldsymbol{q} = [\cos 45°, 0, 0, \sin 45°] = [0.707, 0, 0, 0.707]$.

The BNO085 handles all quaternion arithmetic internally and outputs a pre-filtered quaternion, so the user never needs to implement quaternion integration from scratch.

4.5.2 Rotation Matrix from Quaternion

To transform a vector from the body frame (relative to the sensor) into the world frame (fixed to the room), use the rotation matrix:

$$\mathbf{R}(\boldsymbol{q}) = \begin{bmatrix} 1 - 2(q_y^2 + q_z^2) & 2(q_x q_y - q_w q_z) & 2(q_x q_z + q_w q_y) \\ 2(q_x q_y + q_w q_z) & 1 - 2(q_x^2 + q_z^2) & 2(q_y q_z - q_w q_x) \\ 2(q_x q_z - q_w q_y) & 2(q_y q_z + q_w q_x) & 1 - 2(q_x^2 + q_y^2) \end{bmatrix} \tag{34}$$

Each element is a quadratic combination of the quaternion components. This 3×3 matrix rotates any body-frame vector into world coordinates.

4.5.3 Quaternion Kinematics (How Orientation Updates)

If the gyroscope reports a body angular rate $\boldsymbol{\omega}^b = [\omega_x, \omega_y, \omega_z]^\top$ (in rad/s), the quaternion evolves according to:

$$\dot{\boldsymbol{q}} = \frac{1}{2}\boldsymbol{q} \otimes \begin{bmatrix} 0 \\ \boldsymbol{\omega}^b \end{bmatrix} \tag{35}$$

where $\otimes$ is the Hamilton product (a specific rule for multiplying two quaternions, analogous to cross-product for vectors). The 0 in the first slot makes the combined object a *pure quaternion* compatible with the product rule.

For a finite time step Δt, the exact discrete integration is:

$$\boldsymbol{q}_{k+1} = \boldsymbol{q}_k \otimes \begin{bmatrix} \cos\left(\frac{\Delta t}{2}\left\|\boldsymbol{\omega}_k^b\right\|\right) \\ \frac{\boldsymbol{\omega}_k^b}{\|\boldsymbol{\omega}_k^b\|}\sin\left(\frac{\Delta t}{2}\left\|\boldsymbol{\omega}_k^b\right\|\right) \end{bmatrix} \tag{36}$$

This rotates the current orientation by the incremental rotation corresponding to spinning at rate $\boldsymbol{\omega}^b$ for time Δt. The magnitude of the rotation angle is $\left\|\boldsymbol{\omega}_k^b\right\| \Delta t$ radians.

4.5.4 IMU-Driven Position Propagation

The BNO085 also outputs linear acceleration $\boldsymbol{a}^b$ (acceleration with gravity removed) measured in the body frame. To integrate it into world-frame position, first rotate it:

$$\boldsymbol{a}^w = \mathbf{R}(\boldsymbol{q})\,\boldsymbol{a}^b$$

Then apply Newton's equations over one IMU time step $\Delta t = 10\,\text{ms}$ (at $100\,\text{Hz}$):

$$\boldsymbol{p}_{k+1} = \boldsymbol{p}_k + \boldsymbol{v}_k\,\Delta t + \tfrac{1}{2}\boldsymbol{a}_k^w\,\Delta t^2 \tag{37}$$

$$\boldsymbol{v}_{k+1} = \boldsymbol{v}_k + \boldsymbol{a}_k^w\,\Delta t \tag{38}$$

Between two UWB sweeps (300 ms apart) the IMU runs 30 propagation steps, providing smooth position estimates throughout. When the next UWB measurement arrives, the Kalman update corrects the accumulated drift.

4.5.5 Loose vs. Tight Coupling

Loose coupling first runs multilateration to get a position, then feeds that position into the EKF as the measurement $\boldsymbol{z}_k = \hat{\boldsymbol{p}}_{\text{multilat}}$. It is easier to implement but throws away information: the multilateration step collapses the individual range errors into a single covariance, losing the directional information about which anchors were reliable.

Tight coupling feeds raw UWB ranges $\hat{d}_i$ directly into the EKF/MHE as separate measurements via the model of eq. (22). This is the recommended approach because:

- It can use partial anchor sets (e.g. 2 of 4 anchors visible) without changing any equations.
- Each anchor's quality metric flows directly into the measurement weight $\mathbf{R}_k$.
- The Jacobian $\mathbf{H}_i$ automatically captures the geometry of each anchor.

4.5.6 Roll, Pitch, and Yaw from Quaternion

Roll (ϕ): rotation about the forward axis (tilting left/right like an aircraft banking). **Pitch** (θ): rotation about the side axis (nose up/down). **Yaw** (ψ): rotation about the vertical axis (turning left/right on a compass heading). Together these three angles fully describe orientation in 3D.

The Euler angles in ZYX convention are extracted from the quaternion by:

$$\phi = \text{atan2}\Big(2(q_w q_x + q_y q_z),\ 1 - 2(q_x^2 + q_y^2)\Big) \tag{39}$$

$$\theta = \arcsin(2(q_w q_y - q_z q_x)) \tag{40}$$

$$\psi = \text{atan2}\Big(2(q_w q_z + q_x q_y),\ 1 - 2(q_y^2 + q_z^2)\Big) \tag{41}$$

The function $\text{atan2}(y, x)$ returns the angle whose sine is proportional to y and whose cosine is proportional to x, in the correct quadrant. The arcsin for pitch is why pitch is limited to $\pm 90°$ in this convention.

The BNO085 runs its ARVR Stabilised Rotation Vector mode, which fuses accelerometer, gyroscope, and magnetometer internally at 400 Hz. The magnetometer provides an absolute heading reference (like a compass), preventing the yaw drift that gyroscope integration alone would accumulate over time.

4.6 Anchor Self-Survey via Multidimensional Scaling

Normally we assume the anchor positions are known (measured with a tape measure or laser rangefinder and entered into the config file). Any measurement error directly biases all subsequent position estimates.
Anchor self-survey replaces manual measurement: the anchors range to *each other* and a mathematical procedure called Multidimensional Scaling (MDS) recovers their 2D or 3D layout automatically. The analogy: given a table of driving distances between cities, MDS reconstructs a map of those cities without any compass or coordinate system.

4.6.1 Data Collection

With N anchors, there are $\binom{N}{2} = \frac{N(N-1)}{2}$ unique pairs. For 4 anchors this is 6 pairs. Each anchor temporarily becomes a ranging initiator and ranges to every other anchor. Each pairwise distance is averaged over $M = 100$ measurements to reduce noise: the averaged uncertainty is $\sigma_d/\sqrt{M} \approx 5\,\text{cm}/\sqrt{100} = 0.5\,\text{cm}$.

4.6.2 Classical Multidimensional Scaling

Step 1 — Build the squared-distance matrix. Arrange all pairwise distances $\hat{d}_{ij}$ into a symmetric $N \times N$ matrix and square each entry:

$$D_{ij}^{(2)} = \hat{d}_{ij}^2$$

The diagonal entries are zero (distance from any anchor to itself is zero).

Step 2 — Double-centering. The matrix $\mathbf{D}^{(2)}$ encodes distances relative to the centroid of the anchor cloud, but it is not centred. Apply the centering matrix $\mathbf{J} = \mathbf{I}_N - \frac{1}{N}\mathbf{1}\mathbf{1}^\top$ (which subtracts row and column means):

$$\mathbf{B} = -\tfrac{1}{2}\,\mathbf{J}\,\mathbf{D}^{(2)}\,\mathbf{J} \tag{42}$$

If the anchor positions are stored in a matrix $\mathbf{X}$ (rows = anchors, columns = coordinates), then the squared distances satisfy $D_{ij}^{(2)} = \|\boldsymbol{x}_i - \boldsymbol{x}_j\|^2 = \|\boldsymbol{x}_i\|^2 - 2\boldsymbol{x}_i \cdot \boldsymbol{x}_j + \|\boldsymbol{x}_j\|^2$. The double-centering operation eq. (42) algebraically cancels the $\|\boldsymbol{x}_i\|^2$ and $\|\boldsymbol{x}_j\|^2$ terms, leaving $B_{ij} = \boldsymbol{x}_i \cdot \boldsymbol{x}_j$ — the dot products between anchor positions. In matrix form: $\mathbf{B} = \mathbf{X}\mathbf{X}^\top$. We have converted distances into dot products, from which positions can be extracted by factorisation.

Step 3 — Eigendecomposition. Factor $\mathbf{B}$ into eigenvectors and eigenvalues: $\mathbf{B} = \mathbf{V}\mathbf{\Lambda}\mathbf{V}^\top$. If the distances were exact, exactly d eigenvalues would be non-zero (one per spatial dimension). Take the d largest:

$$\hat{\mathbf{X}} = \mathbf{V}_d \, \mathbf{\Lambda}_d^{1/2} \tag{43}$$

where $\mathbf{V}_d$ contains the d leading eigenvectors as columns and $\mathbf{\Lambda}_d = \mathrm{diag}(\lambda_1, \ldots, \lambda_d)$. Each row of $\hat{\mathbf{X}}$ is the recovered position of one anchor.

Numerical example: 3 anchors in 2D

Suppose the true anchor positions are $\boldsymbol{a}_1 = (0, 0)$, $\boldsymbol{a}_2 = (3, 0)$, $\boldsymbol{a}_3 = (1.5, 2.6)$ metres. The true pairwise distances are: $d_{12} = 3\,\mathrm{m}$, $d_{13} = 3\,\mathrm{m}$, $d_{23} = 3\,\mathrm{m}$ (equilateral triangle, side 3 m).

$$\mathbf{D}^{(2)} = \begin{bmatrix} 0 & 9 & 9 \\ 9 & 0 & 9 \\ 9 & 9 & 0 \end{bmatrix}$$

After double-centering: $\mathbf{B} = \begin{bmatrix} 6 & -3 & -3 \\ -3 & 6 & -3 \\ -3 & -3 & 6 \end{bmatrix}$ (scaled by 1/3 internally). Eigendecomposition yields 2 non-zero eigenvalues. Taking the top 2 and applying eq. (43) recovers the anchor layout up to rotation/reflection. After frame fixation, the output matches the true positions.

4.6.3 Coordinate Frame Fixation

MDS recovers the shape but not the orientation or position (the solution is ambiguous up to rigid-body motion). Three conventions fix a unique coordinate frame:

1. **Translation:** Subtract the position of anchor 1 from all positions, so anchor 1 becomes the origin $(0, 0)$ or $(0, 0, 0)$.
2. **Rotation:** Rotate the layout so that anchor 2 lies on the positive X axis.
3. **Reflection:** If anchor 3 ends up below the X axis (negative Y), flip the Y axis.

The result is written to `config/anchors.json` for immediate use by the localisation pipeline.

4.6.4 Noise Propagation

Each pairwise distance is the mean of $M = 100$ measurements. If individual measurement noise is $\sigma_d \approx 5\,\mathrm{cm}$, the mean has standard deviation $\sigma_d/\sqrt{M} = 0.5\,\mathrm{cm}$. Classical MDS propagates this to a position uncertainty of roughly 0.5 cm per anchor coordinate — within our $\leq 2\,\mathrm{cm}$ self-survey target.

5 Implementation Roadmap

The system is developed in six phases, each independently testable:

Table 3: Implementation phases and key items

Phase	Item	Description	Difficulty
1 — Foundation	1.0	Switch to 64 MHz PRF (accuracy mode)	Easy
	1.1	Better calibration (200 samples, multi-distance)	Easy
	1.2	Adaptive polling: skip dead anchors	Easy
	1.3	Faster sweep rate (reduce reply delay)	Medium
	1.5	Anchor self-survey + auto `anchors.json`	Medium
2 — Ranging	2.1	FP_POWER NLOS detection (firmware + wire format)	Medium
	2.2	Weighted multilateration (NLOS score)	Medium
	2.3	NLOS range bias correction slider	Easy
	2.4	Per-anchor diagnostics panel	Easy
	2.5	GPS-style δ-solve (4+ anchors)	Medium
3 — Estimation	3.1	Constant-acceleration EKF	Medium
	3.2	δ as EKF state (bias estimation)	Med-Hard
	3.3	Moving Horizon Estimator	Hard
	3.4	Hybrid EKF + MHE	Medium
4 — 3D + RPY	4.1	4th anchor + 3D configuration	Easy
	4.2	BNO085 SensorImu (SPI)	Medium
	4.3	RPY output from quaternion	Easy
	4.4	Loose coupling: IMU-aided EKF	Hard
	4.5	Tight coupling: raw ranges + IMU in MHE	Very Hard
5 — Hardening	5.1	Deep sleep on anchors	Medium
	5.2	DW1000 driver error handling	Medium
	5.3	Temperature compensation	Medium
	5.4	Online self-calibration	Hard
6 — Scale	6.1	Multi-tag TDMA superframe	Hard
	6.2	Differential timestamp encoding	Hard
	6.3	On-device 2D solve (OLED)	Medium

6 Expected Performance

The theoretical floor for the DW1000 chip is approximately 1–2 cm in perfect LOS conditions (limited by the 15.65 ps timestamp resolution and residual crystal asymmetry). Practical indoor environments with multipath and occasional NLOS will see 2–5 cm as the sustainable floor. IMU integration does not improve UWB ranging accuracy but eliminates position lag between sweeps and provides orientation at sensor rate.

Table 4: Expected accuracy progression through implementation phases

After Phase	**XY accuracy**	**Z accuracy**	**RPY**	**Rate**	**Key enabler**
Current	3–10 cm	—	—	3 Hz	Baseline
Phase 1	2–6 cm	—	—	4–8 Hz	Calibration + PRF
Phase 2	1–4 cm	—	—	4–8 Hz	NLOS detection
Phase 3	1–3 cm	—	—	4–8 Hz	MHE + CA-EKF
Phase 4	1–3 cm	2–5 cm	3–5°	100 Hz RPY	IMU fusion
Phase 5	1–3 cm	2–5 cm	3–5°	100 Hz RPY	Reliability